```
import heapq
def a_star_search(graph, heuristics, start, goal):
    open_list = [(heuristics[start], 0, start, [start])]
    visited = set()
    while open_list:
        f, g, current, path = heapq.heappop(open_list)
        if current == goal:
            return path, g
        visited.add(current)
        for neighbor, cost in graph[current]:
            if neighbor not in visited:
                g_new = g + cost
                f_new = g_new + heuristics[neighbor]
                heapq.heappush(open_list, (f_new, g_new,
neighbor, path + [neighbor]))

    return None, float('inf')
graph = {
    'a': [('b', 4), ('c', 3)],
    'b': [('f', 5), ('e', 12)],
    'c': [('d', 7), ('e', 10)],
    'd': [('e', 2)],
    'e': [('z', 5)],
    'f': [('z', 16)],
    'z': []
}
heuristics = {
    'a': 14,
    'b': 12,
    'c': 11,
    'd': 6,
    'e': 4,
    'f': 11,
    'z': 0
}
path, cost = a_star_search(graph, heuristics, 'a', 'z')

print(f"Path found: {' -> '.join(path)}")
print(f"Total cost: {cost}")
```